Algoritmusok és adatszerkeztek I.

11. gyakorlat

Tartalom

Hasítás, hasító táblák

- Keresés – tanult adatszerkezetek összehasonlítása
- Hasítás elve
- Hasító függvények
- Láncolt hasító tábla
- Nyílt címzés módszere
- Lineáris próbasorozat
- Négyzetes próbasorozat
- Kettős hasítás

Hasítás, hasító táblák

- Céljuk: hatékony keresés, módosítás (beszúrás, törlés) viszonylag állandónak tekinthető kulcskészleten (kulcsok egyediek)
- Adatszerkezetek eddig:

1. Rendezetlen tömb (lista) + lineáris keresés O(n)
2. Rendezett tömb + logaritmikus keresés O(log n) (módosítások: O(n))
3. Rendezett lista + hatékonyabb lineáris keresés O(n) (módosítások: O(n))
4. Bináris keresőfa: keresés, módosítások a fa magasságával arányosak, ami log n..n intervallumban lehet.
5. Hasítás: keresés jó hatékonyságú hasító tábla esetén konstans lépésszámú (akár 1-2 lépés) (módosító műveletek is hasonlóan hatékonyak).

Hasítás, hasító táblák

Jelölések:

m : a hasító tábla mérete

$T[0..m)$: a hasító tábla

$T[0], T[1], \ldots, T[m-1]$: a hasító tábla rései (slot-jai)

$\oslash$: üres rés a hasító táblában (direkt címzésnél és a kulcsütközések láncolással való feloldása esetén)

E : üres rés kulcsa a hasító táblában (nyílt címzésnél)

D : törölt rés kulcsa a hasító táblában (nyílt címzésnél)

n : a hasító táblában tárolt adatok száma

$\alpha = n/m$: a hasító tábla kitöltöttségi aránya (load factor)

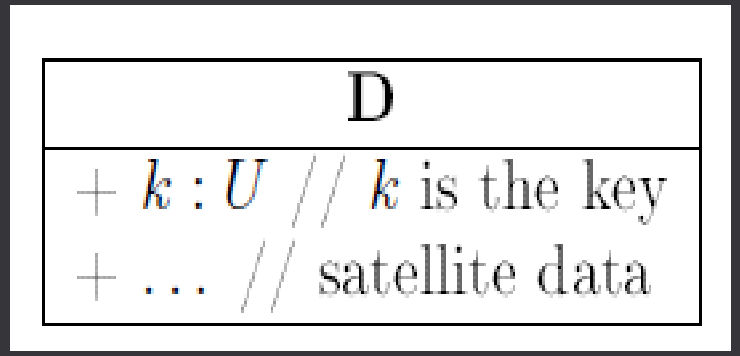
U : a kulcsok univerzuma; $k, k', k_i \in U$

$h : U \to 0..(m-1)$: hasító függvény

Feltesszük, hogy a hasító tábla nem tartalmazhat két vagy több azonos kulcsú elemet, és hogy $h(k)$, $\Theta(1)$ időben számolható.

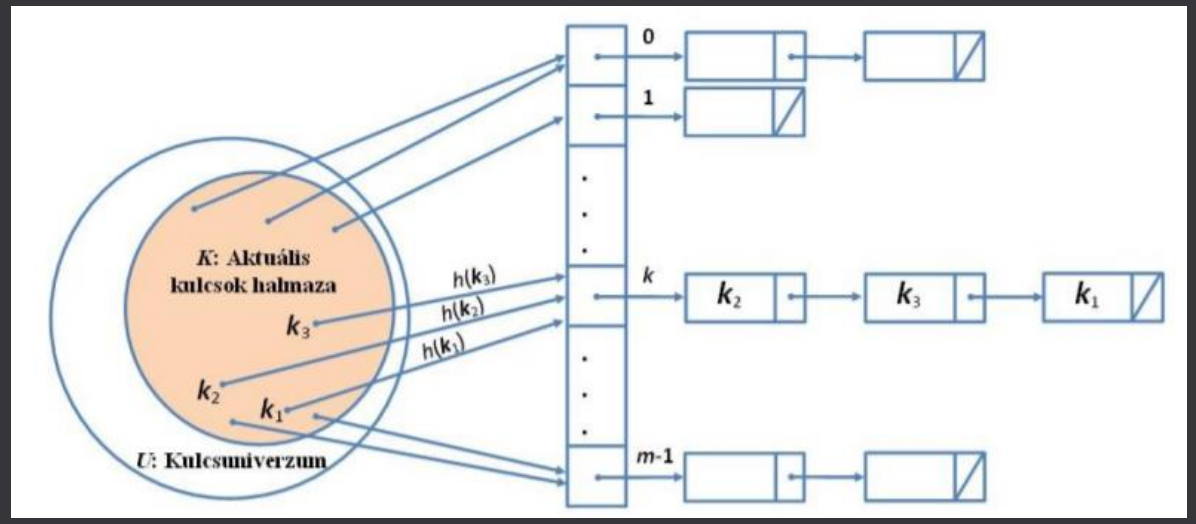
Direkt címzés

- Ha a kulcshalmaz nem túl nagy
- Ha U = [0..m), ahol m≥n, de m nem túl nagy, akkor egy m méretű tömbben, hasítótáblában tárolhatjuk a rekordokra mutató pointereket úgy, hogy a tömb k indexű tagja a k kulcsú rekordra mutat.
- Ha nincs k kulcsú rekord, akkor a pointer értéke ∅.
- T:D*[*m*]



Hasítás jelensége

- m résből álló T[m] hasítótábla
- U – kulcsuniverzum |U|>> n
- h: U $\rightarrow$ 0..m-1 hasító függvény
  - |U|>> m
- A k kulcsú rekordot a hasító tábla T[h(k)] résében (rekeszében) helyezzük el: T[0..m-1] indexű.



- Ha k_1, k_2 kulcsokra $h(k_1) = h(k_2)$, akkor kulcsütközés
- Kulcsütközés feloldása jellemzően két módon: láncolt hasító tábla, nyílt címzéses hasító tábla
- Például, ha k egy nem negatív egész, akkor h(k) = k mod m egy lehet egy hasító függvény.

Hasító függvény

- A h : U → [0..m) függvény egyszerű egyenletes hasítás, ha a kulcsokat a rések között egyenletesen szórja szét, azaz hozzávetőleg ugyanannyi kulcsot képez le az m rés mindegyikére. Tetszőleges hasító függvénnyel kapcsolatos elvárás, hogy egyszerű egyenletes hasítás legyen.
- Ha a kulcsok nem természetes számok:
  - Természetes számként kell megjeleníteni azokat
  - Karaktersorozat kulcsok lehetnek megfelelő számrendszerben felírt egészek
  - pt kulcs → (112, 116) számpár
    - ASCII: p – 112, t – 116
  - 128-as számrendszerben: (112*128) + 116 = 14452

Hasító függvény - Osztó módszer

- Hash függvény: h(k) = k mod m
  - A hash tábla mérete: m = 12, a kulcs: k = 100, akkor h(k) = 4
- Egyenletessége nagyon függ az m megválasztásától, érdemes prímnek választani (ami távol van a szomszédos kettő hatványoktól)
- Ha például a kulcsütközést láncolással szeretnénk feloldani, és kb. 2000 rekordot szeretnénk tárolni α = 3 kitöltöttségi aránnyal, akkor a 701 jó választás
  - A 701 olyan prímszám, ami közel esik a 2000/3-hoz, de a szomszédos kettő hatványoktól, az 512-től és az 1024-től is elég távol van
- Tábla kitöltöttségi aránya: α=n/m (n rekordok száma a táblában, m-tábla réseinek száma). Láncolt hasító tábla esetén ez ad becslést a listák hosszára, ha egyenletes a hasítás, a listák várható hossza: α

Hasító függvény – Kulcsok a [0, 1) intervallumból

- Ha egyenletesen oszlanak el, a

$$h(k) = \lfloor k * m \rfloor$$

függvény is kielégíti az egyszerű, egyenletes hasítás feltételét.

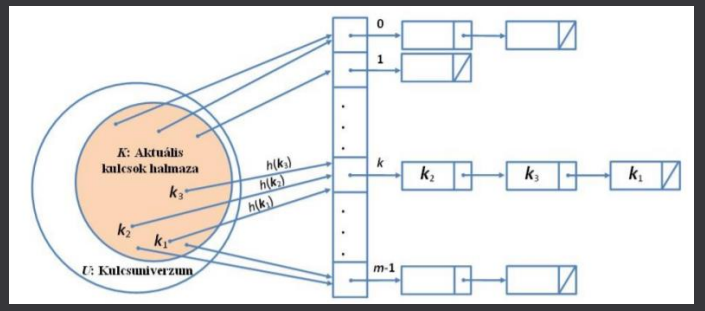
Hasító függvény – Szorzó módszer

- Legyen $0 < A < 1$ egy konstans (ennek megválasztása nehéz feladat).
- Vegyük a k*A érték törtrészét: $0 \leq \{k*A\} < 1$
- $h(k) = \lfloor \{k*A\} * m \rfloor$ $h(k) \in [\, 0..\, m\text{-}1]$ azonnal látszik.
- m megválasztása kevésbé jelentőségteljes az egyenletes hasítás eléréséhez.
- Érdekesség: D. E. Knuth javaslata az A konstans megválasztására:

$$A \approx \frac{\sqrt{5}-1}{2} \approx 0{,}6180339887$$

Láncolt hasító tábla algoritmusai a jegyzetből

- Rések listái S1L listák



E1
$+key : U$ $\ldots$ // satellite data may come here $+next$: **E1***
$+$**E1**() { $next := \oslash$ }

init(T:E1*[m])

$i := 0$ to $m-1$

$T[i] := \oslash$

search(T:E1*[] ; k:U):E1*

return searchS1L($T[h(k)]$,k)

searchS1L(q:E1* ; k:U):E1*

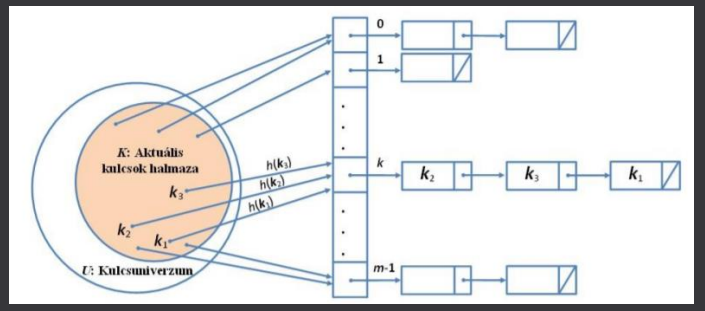
$q \neq \oslash \wedge q \rightarrow key \neq k$

$q := q \rightarrow next$

return q

Láncolt hasító tábla algoritmusai a jegyzetből

- Rések listái S1L listák



E1
$+key : U$
... // satellite data may come here
$+next$: **E1***
+E1() { $next := \oslash$ }

searchS1L(q:E1* ; k:U):E1*

$q \neq \oslash \wedge q \rightarrow key \neq k$

$q := q \rightarrow next$

return q

insert(T:E1*[] ; p:E1*):$\mathbb{B}$

$k := p \rightarrow key$; $s := h(k)$

searchS1L$(T[s], k) = \oslash$

$p \rightarrow next := T[s]$	return $false$
$T[s] := p$	
return $true$	

remove(T:E1*[] ; k:U):E1*

$s := h(k)$; $p := \oslash$; $q := T[s]$

$q \neq \oslash \wedge q \rightarrow key \neq k$

$p := q$; $q := q \rightarrow next$

$q \neq \oslash$

$p = \oslash$

$T[s] := q \rightarrow next$	$p \rightarrow next := q \rightarrow next$	SKIP
$q \rightarrow next := \oslash$		

return q

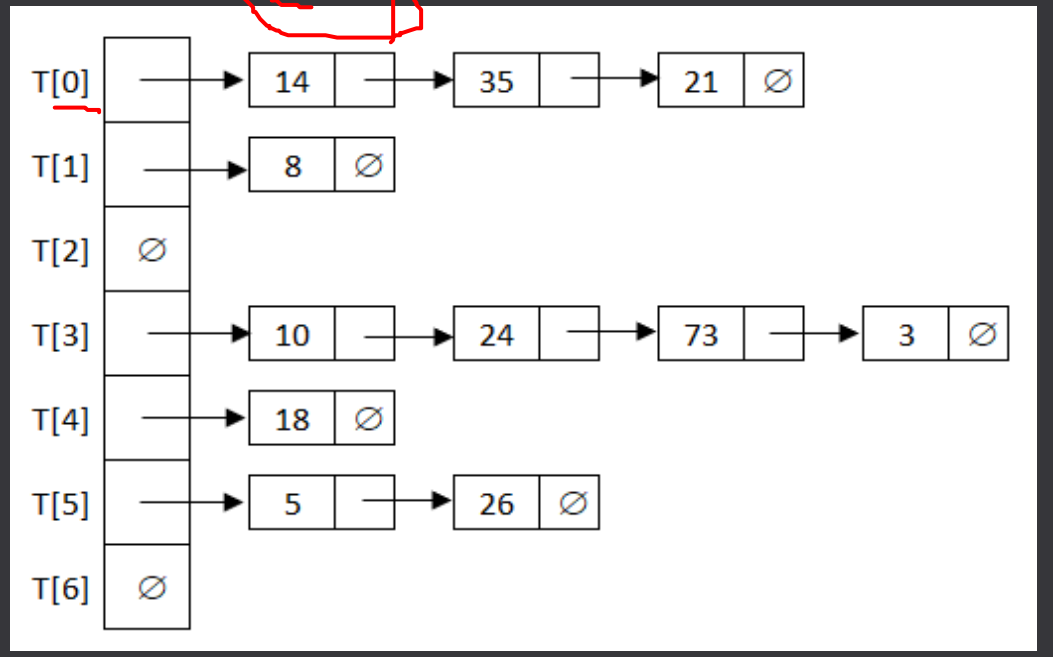
Láncolt hashelés

- Példa: m = 7, h : N→0..6, h(k) = k mod 7.
- A táblába beszúrt elemek sorban:

3, 21, 73, 8, 24, 35, 26, 14, 5, 10, 18.

- A kapott hasító tábla a listákkal:

k	h(k)
3	3
21	0
73	3
8	1
24	3
35	0
26	5
14	0
5	5
10	3
18	4



Műveletigények

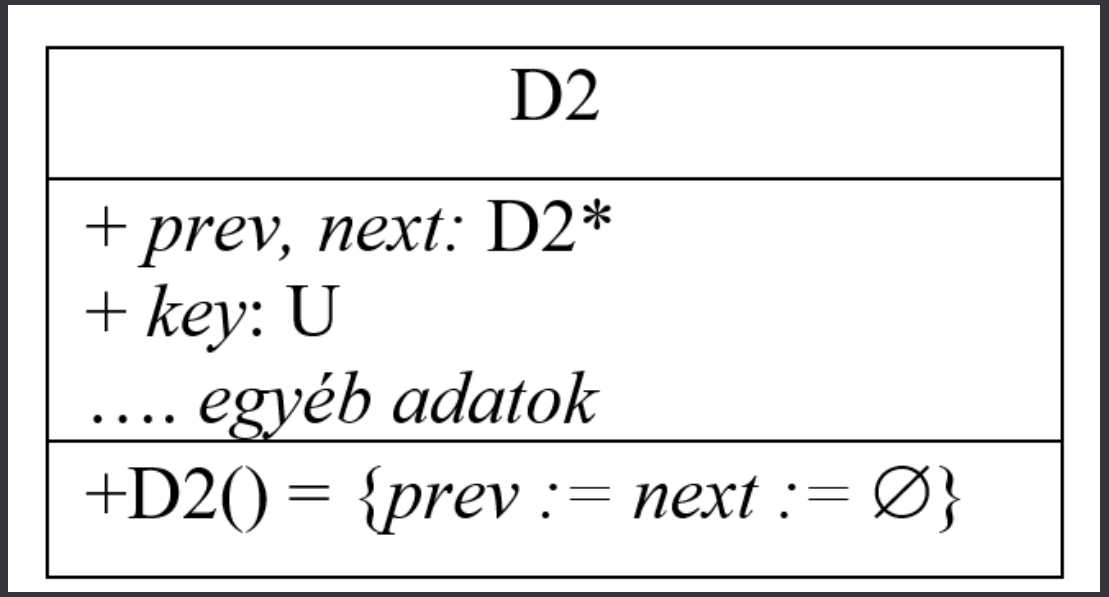
Nyilván $T_{\text{init}}(m) \in \Theta(m)$. A másik három műveletre pedig $mT \in \Theta(1)$, $MT(n) \in \Theta(n)$, $AT(n,m) \in \Theta(1+\frac{n}{m})$.

$AT(n,m) \in \Theta(1+\frac{n}{m})$ feltétele, hogy a $h : U \to 0..(m-1)$ függvény *egyszerű egyenletes hasítás* legyen, és még az indokolja, hogy a résekhez tartozó listák átlagos hossza $= \frac{n}{m} = \alpha$.

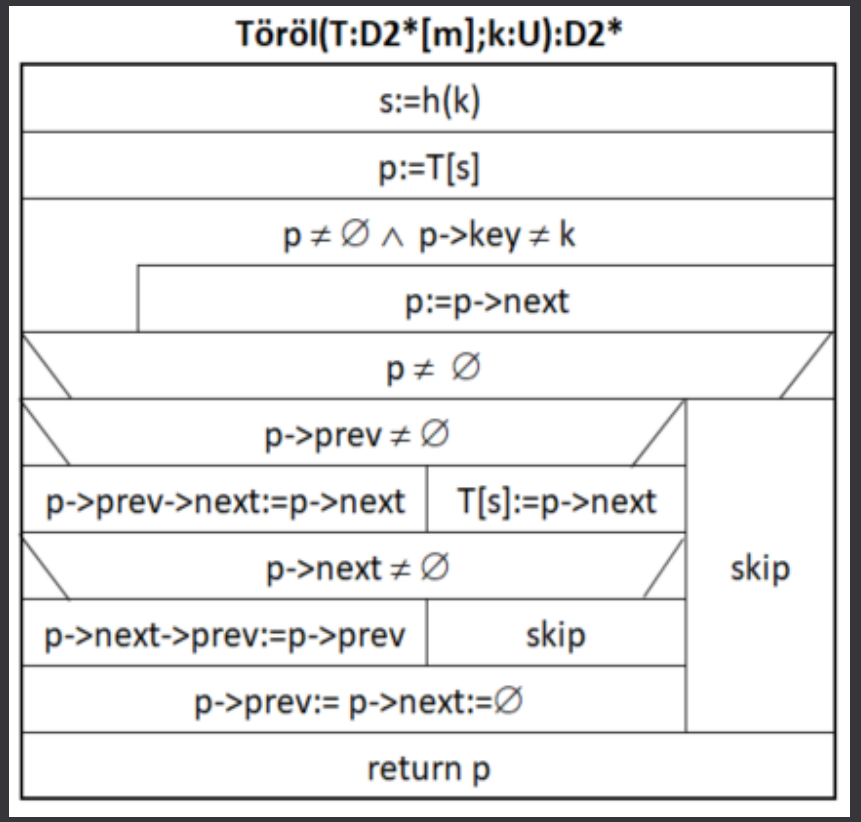
Általában feltesszük még, hogy $\frac{n}{m} \in O(1)$. Ebben az esetben nyilván $AT(n,m) \in \Theta(1)$ is teljesül.

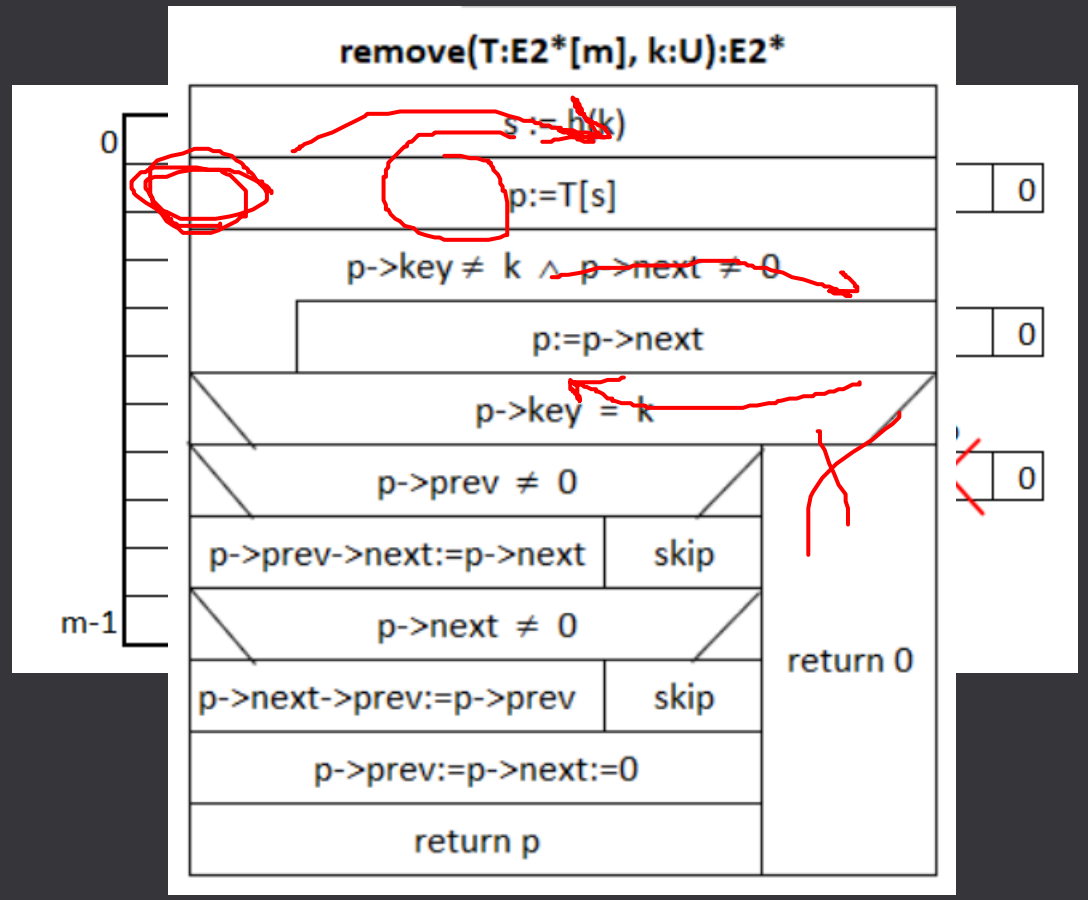
Törlés

- Készítsük el a törlés művelet struktogramját! A listák legyenek kétirányú, fejelem nélküli, nem ciklikus listák (S2L).
- A listaelemek típusára bevezetjük a D2 elemtípust:



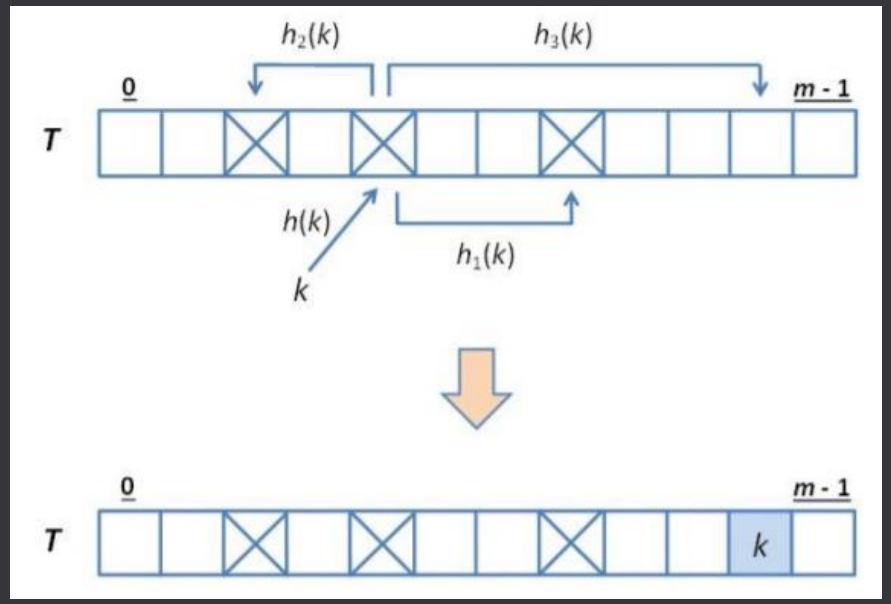
Törlés kétirányú lánc esetén





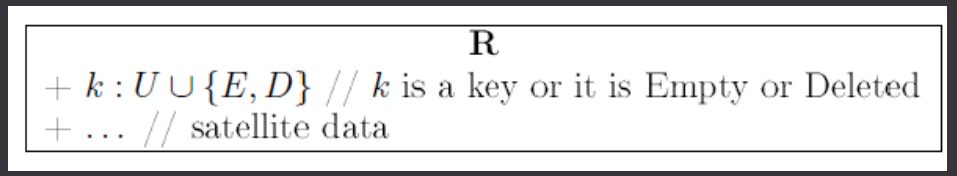
Nyíltcímzéses módszerek

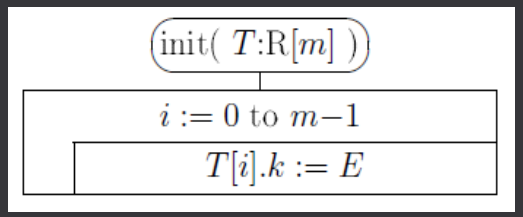
- A rekordok közvetlenül a táblában lesznek elhelyezve.
- Beszúrásnál, ha az adott rés már foglalt, egy próbasorozat mentén megpróbálunk helyet találni az elemnek.
- A próbasorozat elemei:
- h(k,0) h(k,1) h(k,2) … h(k,m-1)
- Fontos, hogy a próbasorozat bejárja a teljes táblát, ezért a próbasorozat értékei a 0..m-1 egészek egy permutációja kell legyen.
- Fontos az is, hogy könnyen reprodukálható legyen. (Adott kulcs esetén mindig ugyanazt a próbasorozatot kell kapjuk.)



Szemléltető ábra dr Fekete István jegyzetéből.
h(k)=h(k,0), h_1(k)=h(k,1)
h_2(k)=h(k,2)

Jelölések, fogalmak a nyílt címzéshez

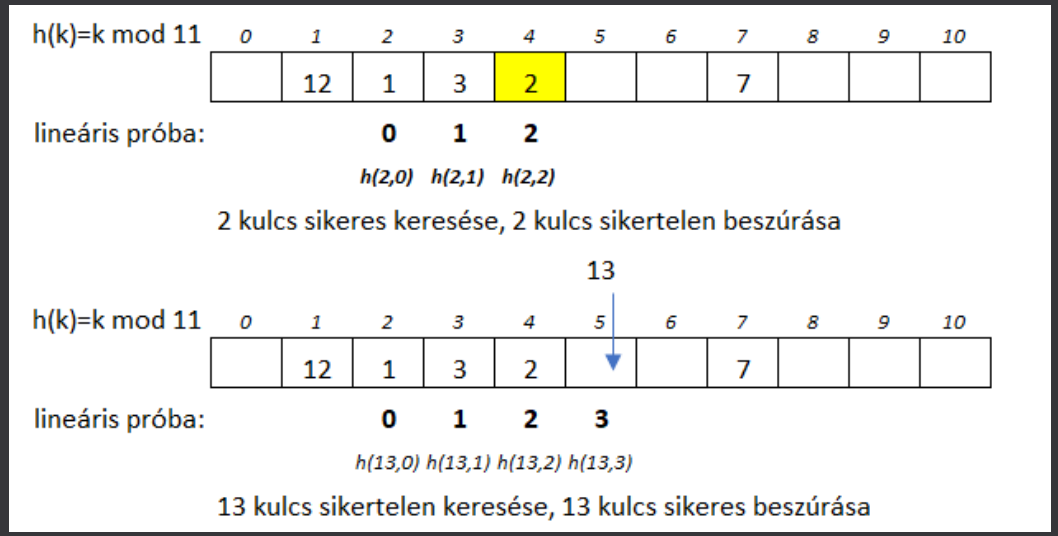




- T:R[m]
- k egy kulcs,vagy
  E –üres a rés D- törölt a rés. E,D extremális konstansok: E,D ∉ U
- Jelölések:
  - h:U x 0..(m-1) → 0..(m-1) : hasító próba
    ⟨h(k,0),h(k,1),h(k,2), … h(k,m-1)⟩ : potenciális próbasorozat
- Az üres és törölt rések a szabad rések, a többi foglalt rés.
- Aktuális próbasorozat: a potenciális próbasorozat egy prefixe.
- m darab hasító függvényünk van: $h(\cdot, i) : U \to 0..(m-1) \qquad (i \in 0..(m-1))$
- Ideális esetben egy tetszőleges potenciális próbasorozat a ⟨0..m-1⟩ sorozatnak mind az m! permutációját azonos valószínűséggel állítja elő. Ilyenkor egyenletes hasításról beszélünk.
- Nevezetes próbasorozatok:
  - Lineáris, négyzetes és kettős hash *(ez utóbbi tekinthető egyenletesnek)*

Nyílt címzés esetén a műveletek

- Elsőként nézzük a műveletek algoritmusait arra az esetre, hogy a táblából nem törölünk.
- **Keresés:** a potenciális próbasorozat szerint haladunk, és keressük a kulcsot. Sikeres keresés esetén megtaláljuk a kulcsot. Ha üres réshez érünk (vagy elvégeztük az összes próbát) leállunk, ekkor sikertelen keresésről beszélünk.
- **Beszúrás:** a potenciális próbasorozaton haladunk, ha megtaláljuk a kulcsot a beszúrás sikertelen, mert már benne van a táblában a kulcs. Ha üres réshez érünk, már biztosan nem lesz a táblában a kulcs, leállunk és az üres résbe beszúrjuk az elemet. Sikertelen a beszúrás, akkor is, ha nem találtunk üres rekeszt, de elvégeztük az összes próbát, ilyenkor a tábla megtelt.



- Műveletigények egyenletes hasítást és $0 < \alpha < 1$ kitöltöttségi arányt feltételezve:
- Sikeres beszúrás és sikertelen keresés legfeljebb: $\frac{1}{1-\alpha}$, 50%-os kitöltöttség esetén ez 2-őt ad.
- Sikeres keresés és sikertelen beszúrás legfeljebb: $\frac{1}{\alpha} ln \frac{1}{1-\alpha}$, 50%-os kitöltöttségre ez kb. 1,387
- α=0,9 (90%-os) kitöltöttség esetén az elsőre legfeljebb 10, a másodikra kb. 2,559 adódik.

Nyílt címzés esetén a műveletek

- Hogyan módosulnak a műveletek, ha a táblában a törlést is megengedjük:
- **Keresés:** a potenciális próbasorozat szerint haladunk, és keressük a kulcsot. D (tartalmú) törölt rekesz esetén folytatódik a művelet, mintha foglalt lenne a rés. Sikeres keresés esetén megtaláljuk a kulcsot. Ha üres réshez érünk (vagy elvégeztük az összes próbát) leállunk, ekkor sikertelen keresésről beszélünk.
- **Beszúrás:** a potenciális próbasorozaton haladunk, ha megtaláljuk a kulcsot a beszúrás sikertelen, mert már benne van a táblában a kulcs. Ha üres réshez érünk, már biztosan nem lesz a táblában a kulcs, leállunk. A D kulcsú törölt rekeszeket itt is átlépjük, de beszúráskor az első törölt rekeszt meg kell jegyezni (ha volt), mert oda érdemes majd a kulcsot beszúrni.
- **Törlés:** kereséssel indul, ha sikeres volt a keresés, a táblából törlődik a kulcs, majd a rés törölt-re állítódik (kulcsa „D” lesz).



Probléma: nagyszámú törlés következtében elfogynak az üres rekeszek (csak foglalt és törölt lesz), ilyenkor a hatékonyság nagyon elromlik: a táblát újra kell építeni.

Lineáris próbasorozat

- Lineáris próbasorozat: h(k,i) = (h(k,0) + i) mod m i=0..m-1

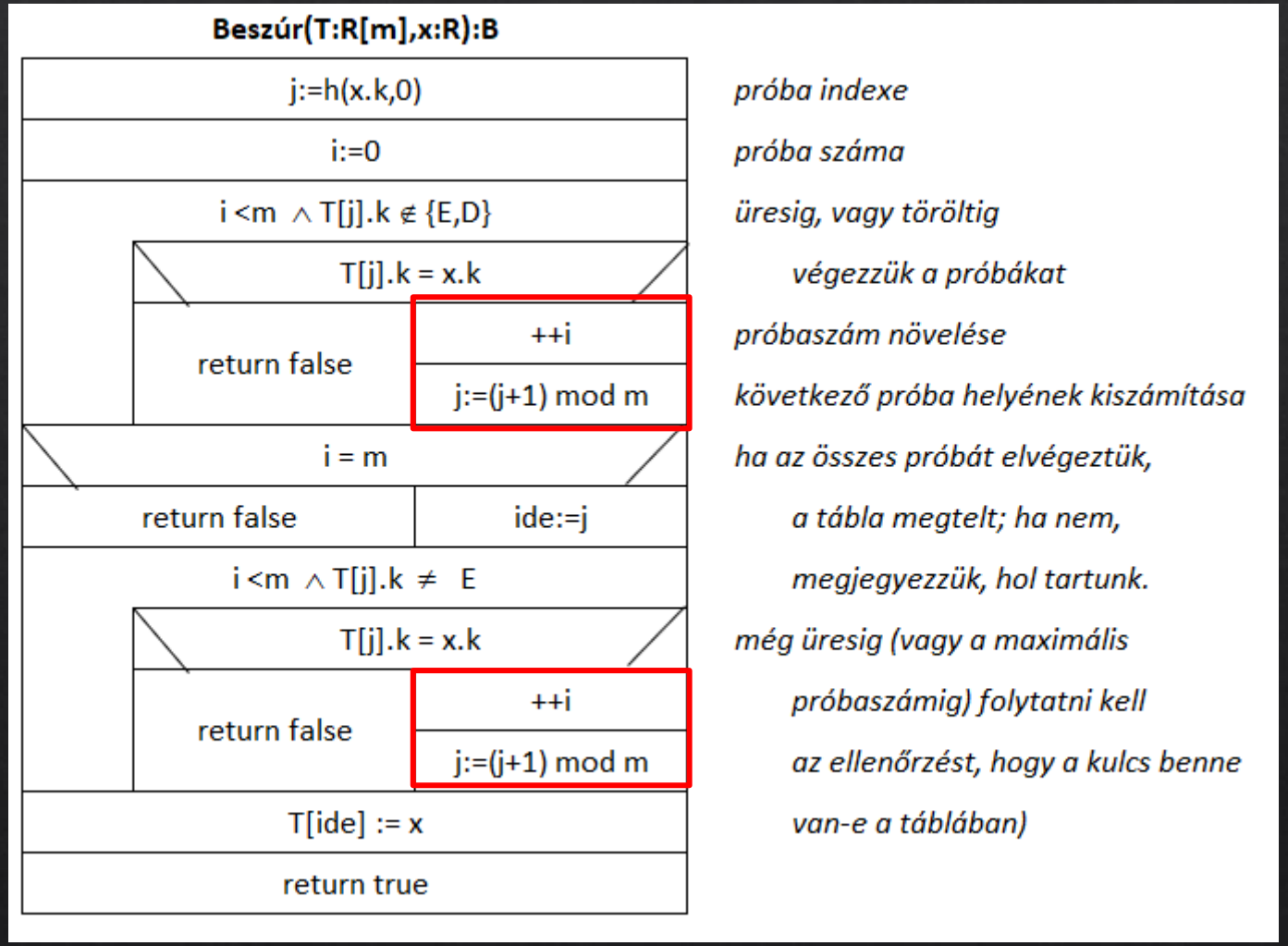
Példa: $m = 11$, $h : \mathbb{N} \to 0..10$, $h(k, n) = (k + n) \bmod 11$.

Művelet	k	$h_1(k)$	Próbasorozat	0	1	2	3	4	5	6	7	8	9	10
Beszúr	24	2	2_E ✓			24								
Beszúr	16	5	5_E ✓			24			16					
Beszúr	57	2	$2, 3_E$ ✓			24	57		16					
Beszúr	32	10	10_E ✓			24	57		16					32
Beszúr	15	4	4_E ✓			24	57	15	16					32
Töröl	57	2	$2, 3_{57}$ ✓			24	D	15	16					32
Beszúr	21	10	$10, 0_E$ ✓	21		24	D	15	16					32
Keres	2	2	$2, 3_D, 4, 5, 6_E$ ✗	21		24	D	15	16					32
Beszúr	2	2	$2, 3_D, 4, 5, 6_E$ ✓	21		24	2	15	16					32
Beszúr	2	2	$2, 3_2$ ✗	21		24	2	15	16					32
Keres	21	10	$10, 0_{21}$ ✓	21		24	2	15	16					32

$\alpha = \frac{6}{11}$

Elsődleges csomósodás jelentkezik, ha a tábla egy szakasza nagyon tele van, pl. 13 beszúrása növeli a 2..5 szakaszban kialakult csomósodás méretét. Ebben a szakaszban megnövekszik az aktuális próbasorozatok hossza.

Beszúrás algoritmusa lineáris próba esetén



Az algoritmus könnyen átírható a másik két próbasorozatra

Négyzetes próba

- $h(k,i) = (h(k,0) + c_1 * i + c_2 * i^2) \bmod m$ alakú a hasító függvény i=0..m-1.
- c_1 és c_2 konstansokat úgy kell megválasztani, hogy a próbasorozat elérje a tábla valamennyi rekeszét ($c1,c2 \in R$, $c2 \neq 0$).
- Ha például m egy 2 hatvány, akkor $c_1=1/2$ és $c_2=1/2$ jó választás.
- $h(k,i) = (h(k,0) + ½ * i + ½ * i^2) \bmod m$, i=0..m-1.
- $h(k,i) = (h(k,0) + 1 + 2 + 3 + \ldots\ldots + i) \bmod m$, i=0..m-1.
- Szokás „álvéletlen” próbának is nevezni.

Négyzetes próbasorozat

Példa: $m = 8$, $h : \mathbb{N} \to 0..7$, $h(k, n) = (k + \frac{n+n^2}{2}) \bmod 8$.

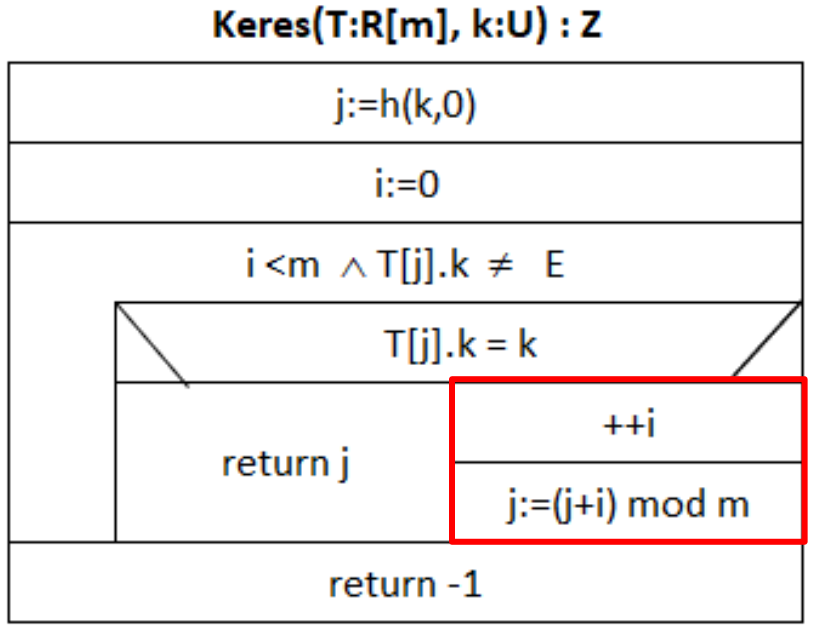
Művelet	k	$h_1(k)$	Próbasorozat	0	1	2	3	4	5	6	7
Beszúr	13	5	5_E ✓						13		
Beszúr	20	4	4_E ✓					20	13		
Beszúr	31	7	7_E ✓					20	13		31
Beszúr	87	7	$7, 0_E$ ✓	87				20	13		31
Beszúr	12	4	$4, 5, 7, 2_E$ ✓	87		12		20	13		31
Töröl	31	7	7_{31} ✓	87		12		20	13		D
Keres	12	4	$4, 5, 7_D, 2_{12}$ ✓	87		12		20	13		D
Keres	15	7	$7_D, 0, 2, 5, 1_E$ ✗	87		12		20	13		D
Beszúr	4	4	$4, 5, 7_D, 2, 6_E$ ✓	87		12		20	13		4
Töröl	10	2	$2, 3_E$ ✗	87		12		20	13		4
Beszúr	35	3	3_E ✓	87		12	35	20	13		4
Beszúr	10	2	$2, 3, 5, 0, 4, 1_E$ ✓	87	10	12	35	20	13		4

$12 + \frac{2+2^2}{2} = 12 + 3 \bmod 8$

$\alpha = \frac{7}{8}$

Másodlagos csomósodás jelentkezik, az azonos próbasorozatok mentén.
Ezeken a helyeken megnövekszik az aktuális próbasorozatok hossza.

Keresés algoritmusa négyzetes próba esetén



próba indexe

próbák száma

Üresig (vagy a maximális próbaszámig)

keressük a kulcsot.

következő próba indexe: mindig eggyel távolabb

ez előző próbától!

Az algoritmus könnyen átírható a másik két próbasorozatra

Más alakú négyzetes próbasorozat

- Kettőhatvány m nem kedvező az osztó módszerhez
- m egy 4d+3 alakú prím (d ∈ N)
- Próbasorozat, ami lefedi a táblát:
  - h(k,0)=h1(k)
  - h(k,1)=h(k,0)-1 mod m, h(k,2)= h(k,0)+1 mod m
  - h(k,3)=h(k,0)-22 mod m, h(k,4)= h(k,0)+22 mod m

- Másodlagos csomósodás ekkor is

Kettős hasítás

- Két hasító függvény van, a lineáris próbához hasonló, de nem +1 az eltolás
- h_1: U $\rightarrow$ 0..m-1, megadja a kulcs helyét a táblában
- h_2: U $\rightarrow$ 1 m-1, ez adja meg a lépések hosszát
- h(k,i) =($h_1(k)$ + i*$h_2(k)$) mod m alakú a hasító függvény i=0..m-1
- $h_2(k)$ hasító függvénnyel kapcsolatos elvárások:
  - $h_2(k)$ legyen nullánál nagyobb, tetszőleges k kulcsra
  - $h_2(k)$ és m legyenek relatív prímek tetszőleges k kulcsra (m-et prímnek választják)
- Jól közelíti az egyenletes hasítást, így kevésbé csomósódik a tábla, hiszen kulcsütközés esetén, azaz ha $h_1(k_1) = h_1(k_2)$, akkor várhatóan $h_2(k_1) \neq h_2(k_2)$ teljesül , és így a kapott potenciális próbasorozat a két elemnél eltérő lesz.

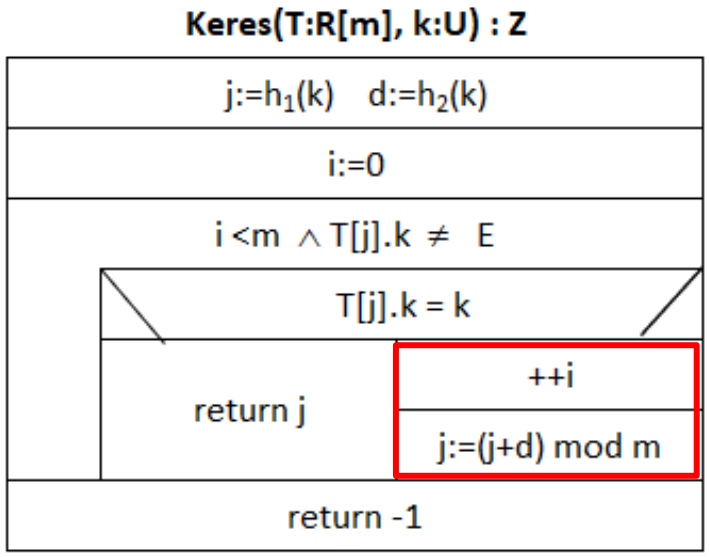
Kettős hasítás próbasorozat

Példa: $m = 11$, $h_1(k) = k \bmod 11, h_2(k) = 1 + (k \bmod 10)$.

Művelet	k	$h_1(k)$	$h_2(k)$	Próbasorozat	0	1	2	3	4	5	6	7	8	9	10
Beszúr	23	1	4	1_E ✓		23									
Beszúr	42	9	3	9_E ✓		23								42	
Beszúr	31	9	2	$9, 0_E$ ✓	31	23								42	
Beszúr	110	0	1	$0, 1, 2_E$ ✓	31	23	110							42	
Beszúr	55	0	6	$0, 6_E$ ✓	31	23	110				55			42	
Töröl	55	0	6	$0, 6_{55}$ ✓	31	23	110				D			42	
Keres	72	6	3	$6_D, 9, 1, 4_E$ ✗	31	23	110				D			42	
Beszúr	14	3	5	3_E ✓	31	23	110	14			D			42	
Töröl	22	0	3	$0, 3, 6_D, 9, 1, 4_E$ ✗	31	23	110	14			D			42	
Töröl	110	0	1	$0, 1, 2_{110}$ ✓	31	23	D	14			D			42	
Beszúr	13	2	4	$2_D, 6_D, 10_E$ ✓	31	23	13	14			D			42	

$\alpha = \frac{5}{11}$

Keresés algoritmusa kettős hasítás esetén



próba indexe, lépés nagysága

próbák száma

Üresig (vagy a maximális próbaszámig)

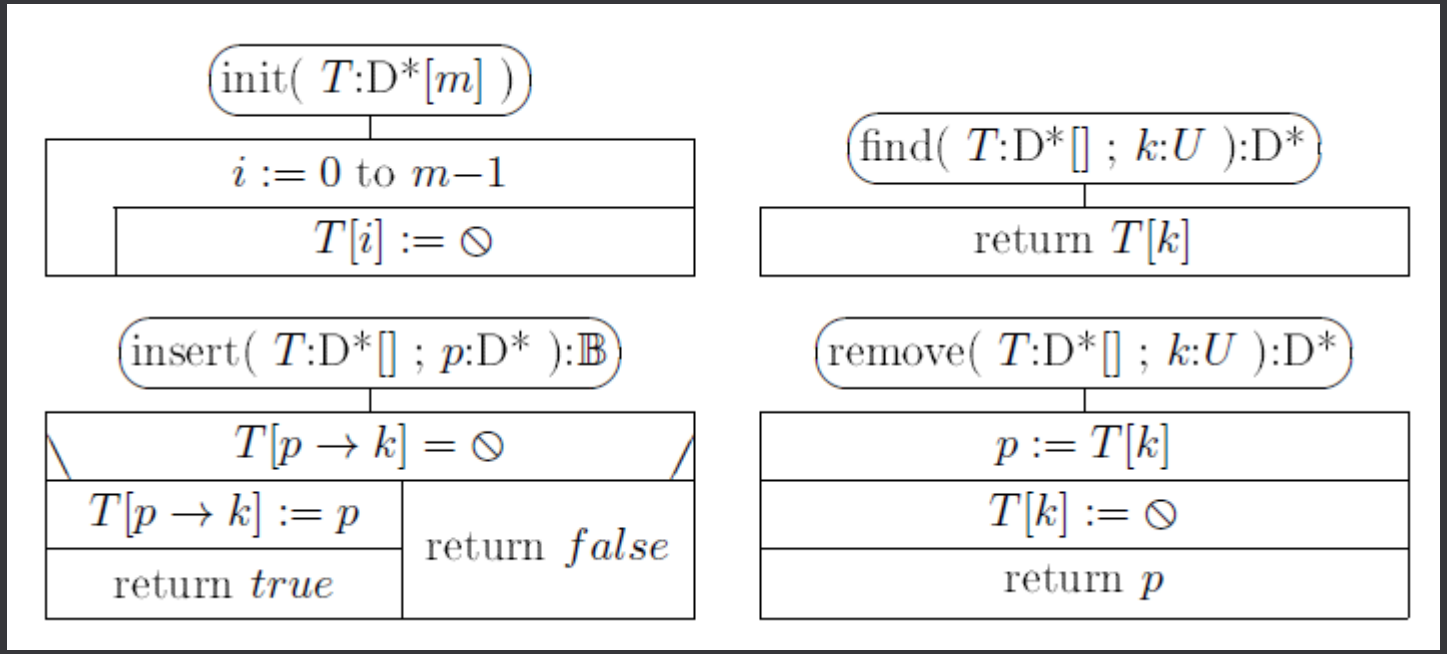
keressük a kulcsot.

következő próba indexe: mindig d-vel távolabb

ez előző próbától!

Direkt címzés (hasítás egyszerű módszere)

- Akkor alkalmazható, ha U: [0..m) és m≥n nem túl nagy. Ekkor a hasító tábla rései pointerek, melyek D típusú elmre mutatnak: T: D*[m]



D
+ $k : U$ // k is the key
+ ... // satellite data

- Műveletigény: init: Θ(n), a többi művelet Θ(1)